



EHR Media Intelligence Platform

Code assessment for AI full-stack intern candidates: Python backend · Tailwind CSS frontend

The Problem

Clinical staff using Epic EHR waste significant time manually reviewing and searching unstructured patient media records — lab PDFs, imaging notes, discharge summaries, scanned documents — that live outside structured fields. Your task is to build an AI-powered pipeline that ingests raw EHR data, cleans and normalizes it to **HL7 FHIR R4**, generates clinical summaries, and surfaces a search interface that lets clinicians find relevant records in seconds.

Task 1: Data Ingestion & Cleaning

Build a Python ingestion pipeline that can ingest messy, heterogeneous EHR records and produce clean, structured output ready for FHIR mapping.

- ☐ Accept at least two input formats: JSON (simulated EHR export) and plain-text/CSV (scanned notes, lab results etc)
- ☐ Handle common data quality issues: missing fields, inconsistent date formats, duplicate records, conflicting patient identifiers
- ☐ Normalize patient demographics (DOB, gender codes, MRN formats) to a canonical schema
- ☐ Output a clean, validated intermediate representation (Pydantic models or dataclasses) with a per-record audit log of changes made
- ☐ Write unit tests covering at least 3 cleaning edge cases

Task 2: FHIR R4 Normalization

Map the cleaned records to valid HL7 FHIR R4 resources using Python.

- ☐ Map at minimum three resource types: Patient, DocumentReference, and DiagnosticReport
- ☐ Use fhir.resources or fhirclient Python library for schema validation
- ☐ Generate a FHIR Bundle (JSON) for each patient with correct resource references (subject, encounter)
- ☐ Validate all Bundles against the FHIR R4 specification; surface validation errors clearly in a report
- ☐ Store Bundles in a local SQLite or file-based store for downstream retrieval



Task 3: AI-Powered Clinical Summarization

Integrate an LLM to generate concise, structured clinical summaries from the FHIR Bundles.

- ☐ Use the Claude API (free tier / Anthropic trial key acceptable) or OpenAI API to summarize each patient's document history
- ☐ Summary must include: chief concern, key diagnoses, recent media records (imaging, labs), and any flagged anomalies
- ☐ Implement prompt engineering to keep summaries under 200 words and clinically accurate
- ☐ Cache summaries to avoid redundant API calls on re-runs (file or SQLite cache keyed by patient ID + record hash)
- ☐ Include a confidence/disclaimer field noting the output is AI-generated and not a clinical decision

Task 4: Semantic Search

Build a search backend that lets clinicians query across all patient records using natural language.

- ☐ Embed all FHIR document text and AI summaries using a sentence-transformer model (e.g., all-MiniLM-L6-v2) or API embeddings
- ☐ Store embeddings in a lightweight vector store: chromadb, FAISS, or SQLite-vss
- ☐ Expose a POST /search FastAPI endpoint that accepts a free-text query and returns the top-5 ranked patient record matches with relevance scores
- ☐ Support filtering by FHIR resource type and date range as query parameters
- ☐ Return results in under 2 seconds for 50 records

Task 5: Frontend Search & Summary UI

Build a clean, clinician-facing web interface using HTML/JS with Tailwind CSS (or React + Tailwind).

- ☐ Search bar that calls POST /search and renders ranked results in real time
- ☐ Result cards displaying: patient name/MRN, record date, resource type badge, relevance score, and AI summary snippet
- ☐ Patient detail drawer/modal showing the full AI summary and a list of linked FHIR resources
- ☐ Filter controls: resource type dropdown, date range picker



- ☐ Responsive layout, loading states, and empty-state messaging
- ☐ Accessible markup (ARIA labels, keyboard navigation for search and results)

Required Stack

Backend	Frontend
• Python 3.11+ • FastAPI • Pydantic v2 • Fhir.resources • Sentence-transformers • chromadb or FAISS • SQLite • pytest	• Tailwind CSS • Vanilla JS or React • Fetch API

Deliverables

GitHub Repository	Short Write-up (max 1 page)
☒ Clean commit history (one commit per major task) ☒ README.md with setup instructions and design decisions ☒ requirements.txt or pyproject.toml ☒ All tests runnable via pytest	☒ Tradeoffs made and why ☒ What you would improve with more time ☒ Any FHIR or clinical concepts you had to research ☒ How you validated AI summary quality

Submission: Email a link to your public GitHub repository and your write-up PDF to Emmanuel @ eobasuyi@onyeone.com by the end of Day, Wednesday, April 16th. Late submissions will not be reviewed. Do not include any real patient data in your repo.